Raising the Bar Without Raising the Budget

October, 2026 · @Lekhyasri Pandavula

Overview

The client asked for "good" quality. By fixing how we built rather than adding to what we built, we delivered measurably better quality on the same budget and deadline. Rework fell from 22% to 7% of team effort, and the client came back to us for a second phase.

A practice case study based on a realistic, fictional scenario.

Aspect	Detail
My role	Project Manager
Project	Customer self-service portal for an external client: account management, appointment booking and secure document upload
Duration	8 months, fixed go-live date
Budget	£480,000 fixed, including a £30,000 contingency reserve
Team	9 people: project manager, business analyst, 4 developers, 2 QA testers, 1 UX designer
Delivery approach	Hybrid: PRINCE2-style stages and governance, delivered in two-week Scrum sprints
Focus	Quality planning, quality tracking and process improvement
Tools	Jira for the defect log, Excel for the quality register and Pareto analysis, Miro for root cause workshops

The situation

The client's acceptance criteria defined "good enough", and they were happy with that bar. Our firm set a higher internal target, because repeat work depends on clients remembering a smooth, low-hassle delivery.

Quality measure	Client's acceptance criteria	Our internal target
Critical defects open at go-live	0	0

Quality measure	Client's acceptance criteria	Our internal target
Minor defects open at go-live	15 or fewer	5 or fewer
UAT test cases passed first time	90%	97%
Rework as a share of team effort	Not specified	Under 10%
Sprint commitments delivered	Not specified	90%

The scope stayed exactly as the client signed it off. Our higher target was about delivering that same scope with fewer defects and less disruption, not adding extras.

Both sets of targets were recorded in the Quality Management Approach at kick-off. What we did not define was the process that would get us there. We assumed the team's existing habits would be enough.

The challenge

At the end-of-stage review in month 3, the quality trend showed we were heading for "barely acceptable", not "impressive". We had logged every defect in Jira since sprint 1, but we had been reviewing counts sprint by sprint, never the trend.

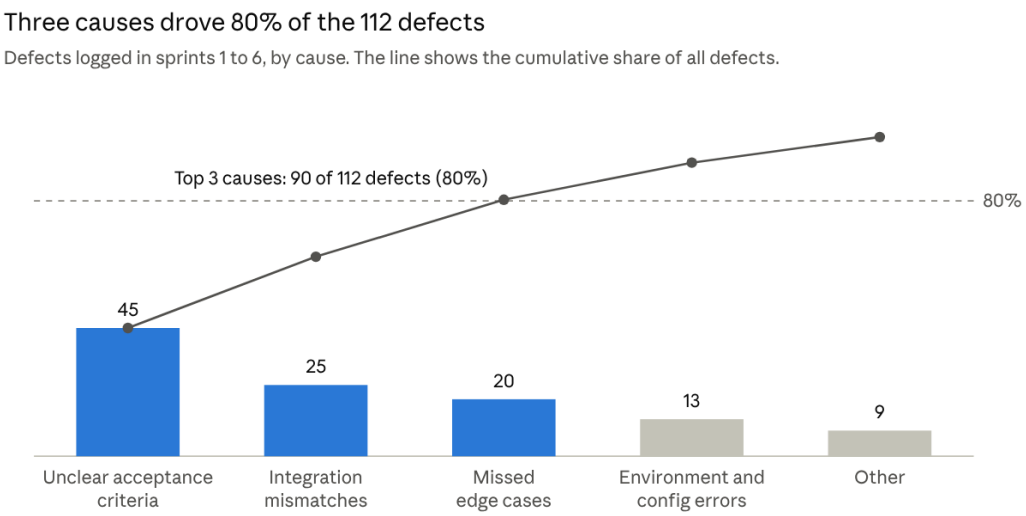
When I plotted the first six sprints together, four problems stood out:

- Defects per sprint had more than doubled, from 12 in sprint 1 to 28 in sprint 6.
- Rework was consuming 22% of team effort, roughly two people's worth of capacity.
- 41% of defects escaped the sprint and were only caught in UAT dry runs with the client.
- Only 78% of UAT test cases passed first time, below even the client's 90% minimum.

If nothing changed, my forecast showed a three-week slip, or paid overtime that would eat most of the contingency. Either outcome would leave the client with a stressful launch, the opposite of what wins repeat work.

Finding the root cause

Three causes produced 80% of the 112 defects, and the biggest one started before any code was written. I ran a Pareto analysis of every defect from sprints 1 to 6, categorized by where it originated.



Illustrative case-study data · defect log, sprints 1 to 6

I then ran a 45-minute 5 Whys workshop with the team on the top cause:

1. Why did so many defects trace back to requirements? Stories entered sprints without clear, testable acceptance criteria.
2. Why? The business analyst wrote stories alone, and developers pulled them in quickly to protect velocity.
3. Why did nobody check stories before a sprint started? We had a Definition of Done, but no Definition of Ready.
4. Why not? Kick-off focused on scope and schedule, so the quality process was assumed rather than designed.
5. Why did that hurt so much? Testers only saw the work after it was built, so every ambiguity surfaced late, when it was most expensive to fix.

The root cause was a process gap, not a people problem: no shared quality standard for work entering a sprint, and testing that started too late.

The improvements

I introduced five process changes; each aimed at a specific cause from the Pareto. Together they cost about 6% of team effort, against the 22% we were losing to rework.

Change	What it involved	Cause it targeted
Definition of Ready with "three amigos" refinement	The business analyst, a developer and a tester reviewed every story before it entered a sprint. Acceptance criteria were	Unclear acceptance criteria (40%)

Change	What it involved	Cause it targeted
	written in Given/When/Then format so they could be tested.	
Sprint integration check-in with the client's technical lead	A shared interface specification and a 30-minute review each sprint. Integration tests ran in a shared test environment from the first week of each sprint.	Integration mismatches (22%)
Peer code review checklist	A second developer reviewed every change against a 10-point checklist covering edge cases, error handling and security basics.	Missed edge cases (18%)
Shift-left testing and regression automation	Testers wrote test cases during refinement, before build started. An automated regression suite covered the 20 most critical user journeys.	Late discovery across all causes
Deployment hand-off checklist	A standard checklist completed before any build moved into the test environment.	Environment and configuration errors (12%)

I ran the changes as a Plan-Do-Check-Act cycle rather than a big-bang rollout:

1. Plan: agreed the five changes and their success measures with the team in a retrospective.
2. Do: piloted them in sprints 7 and 8.
3. Check: defects fell from 28 to 19 per sprint across the pilot, and rework dropped to 15%.
4. Act: kept all five, but cut the code review checklist from 15 points to 10 after developers said it was slowing reviews without catching more issues. Rolled out fully from sprint 9.

Paying for it without raising the budget

The improvements paid for themselves, because poor quality was already costing more than good quality would. This is the cost of quality principle: money spent preventing defects is usually far less than money spent fixing them.

Our team cost was about £56,000 a month. At 22%, rework was costing roughly £12,400 a month to fix work we had already paid to build once. The new practices added about £3,400 a month of prevention effort and cut rework to about £3,900 a month, freeing roughly £5,000 of capacity every month.

The one thing the team could not absorb was building the regression suite. I proposed a contract test automation engineer for six weeks, at £12,000. Quality was already the top risk on the risk register, so this was a planned risk response funded from contingency, not new spending.

Our delivery director, as project executive, approved it within two days. The client paid nothing extra, and we finished at £466,000 of the £480,000 budget.

Bringing the team and the client with me

New process can feel like extra admin, so I treated adoption as a change in its own right.

With the team:

- I shared the Pareto chart in a retrospective and asked the team to design the fixes, rather than handing them a new process.
- I framed the changes around their own frustration: rework meant fixing the same work twice under deadline pressure.
- We piloted for two sprints before committing, and I changed the checklist when the data and the team said it was too heavy.
- The defect trend went up on the sprint board every sprint, so everyone could see the improvement.

With the client:

- At the month-3 stage review, I showed the client sponsor the trend, the root causes and the plan on a single page.
- I confirmed in writing that scope, cost and go-live date would not change.
- The client's technical lead agreed to join the integration check-in, because it cut their own UAT effort.
- I shared the quality dashboard with the client every two weeks, so they watched quality rise rather than hearing about it at the end.

Why this wasn't gold plating

- Gold plating means giving a client extras they never asked for, such as unrequested features or polish. It is a recognized project management anti-pattern: it spends budget without approval, adds risk, and sets expectations the firm may not be able to repeat.
- This project avoided it in three ways:
- The scope never changed. We raised quality without raising grade: the same features, delivered with fewer defects and more reliably.
- The client paid nothing extra. The only additional spend came from contingency, through formal approval, to treat a logged risk.
- Nothing was hidden. The client saw our internal targets and the quality dashboard throughout.
- The client was impressed by a calm, predictable launch, not by surprises.

How I tracked quality

I tracked six measures on a one-page dashboard, reviewed with the team every sprint and shared with the client every two weeks. The lesson from month 3 shaped it: every measure was shown as a trend, never as a single number.

Measure	How it was calculated	Frequency	Owner
Defects per sprint	Defects logged in Jira, split by severity	Every sprint	QA lead
Defect escape rate	Defects found after the sprint they were built in, as a % of all defects	Every sprint	QA lead
Rework effort	Hours logged against defect fixes, as a % of total team hours	Every sprint	Project manager
Sprint commitment delivered	Story points accepted, as a % of points committed	Every sprint	Project manager
Open defects by severity	Critical and minor defects still open in Jira	Weekly	QA lead
UAT first-time pass rate	Test cases passed on first run, as a % of cases run	Each UAT cycle	Business analyst

Every quality check was also recorded in a PRINCE2-style quality register, so the client sponsor could see what had been checked, how, by whom, and the result. A sample from the delivery stage:

ID	Product	Quality method	Responsible	Approver	Planned	Actual	Result
Q-014	Booking flow user stories	Three amigos review against Definition of Ready	Business analyst	QA lead	Sprint 9	Sprint 9	Pass
Q-021	Document upload integratio	Integration test in shared	Lead developer	Client technical	Sprint 10	Sprint 10	Failed, fixed, passed on

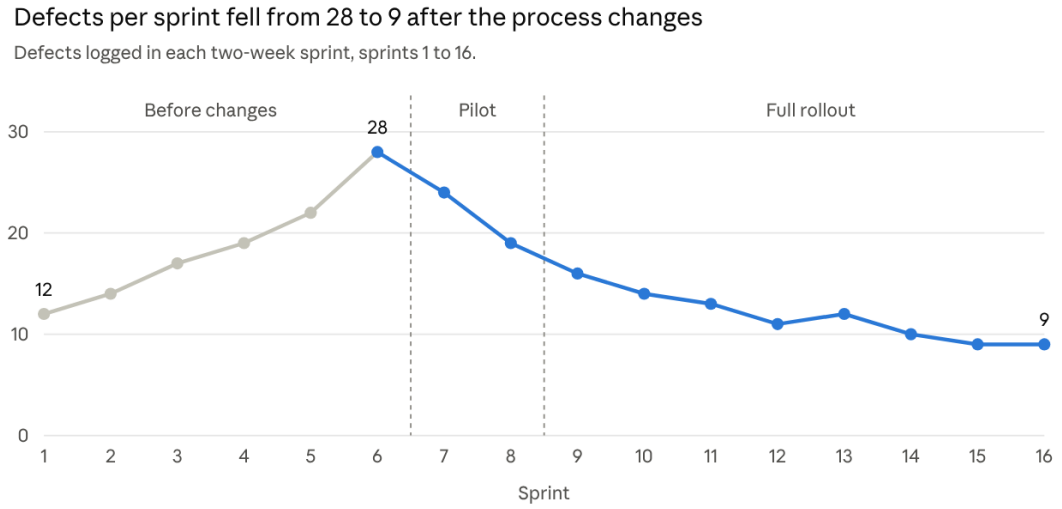
ID	Product	Quality method	Responsible	Approver	Planned	Actual	Result
	n	environment		lead			retest
Q-027	Account management module	Automated regression run, 20 critical journeys	QA tester	QA lead	Sprint 12	Sprint 12	Pass
Q-033	Release candidate build	Deployment hand-off checklist	Developer on rotation	Project manager	Sprint 14	Sprint 14	Pass
Q-040	Full portal	Client user acceptance testing	Client business users	Client sponsor	Sprint 16	Sprint 16	Accepted, 96% passed first time

Results

We went live on the agreed date and under budget. We beat the client's acceptance criteria on every measure and hit four of our five internal targets.

Measure	Month 3	At go-live	Against target
Rework effort	22%	7%	Beat our target of under 10%
Defect escape rate	41%	12%	Cut by more than two-thirds
UAT first-time pass rate	78% in dry runs	96%	Beat the client's 90%; just missed our 97%
Minor defects open	34	4	Beat the client's 15 and our 5
Critical defects open	3	0	Met both
Sprint commitment delivered	70%	92%	Beat our target of 90%

Measure	Month 3	At go-live	Against target
Go-live date	3-week slip forecast	On the agreed date	On time
Spend	On track to use most of contingency	£466,000 of £480,000	Under budget



Illustrative case-study data · defect log, sprints 1 to 16

The portal had only two minor production incidents in its first 30 days. Six weeks after go-live, the client signed a second-phase engagement with us, and named the smooth launch as the reason.

What I learned

- Define the quality process, not just the targets. Next time I would agree a Definition of Ready and a testing approach in the project initiation documentation, before sprint 1.
- Track trends from day one. The data existed from sprint 1, and a simple trend chart would have flagged the problem by sprint 3 rather than month 3.
- Put a price on poor quality. Showing rework in pounds turned a "process" conversation into a business decision the delivery director approved in two days.
- People adopt what they help design. The team owned changes they had shaped, and the pilot gave them room to push back.
- Where I fell short: we missed our 97% UAT target. Most failures sat in the document upload module, which depended heavily on the client's systems. I would bring the client's technical lead in from sprint 1, not sprint 7.

Skills and frameworks demonstrated

Skill	Where it shows up
Quality planning	Separate client acceptance criteria and internal targets, recorded in the Quality Management Approach
Quality tracking and control	Six-measure trend dashboard, Jira defect log, PRINCE2-style quality register
Root cause analysis	Pareto analysis of 112 defects and a 5 Whys workshop
Process improvement	Plan-Do-Check-Act pilot and five targeted changes that removed rework waste
Cost management	Cost of quality case, contingency released through formal approval, finished under budget
Risk management	Quality logged as the top risk, with contingency used as a planned response
Stakeholder management	One-page stage review for the sponsor and a fortnightly client dashboard
Change management	Team co-designed the changes, piloted before rollout, progress kept visible
Agile delivery	Definition of Ready, three amigos refinement, retrospectives and shift-left testing